settings. We can update the dynamic settings using the `_settings` endpoint, as shown in the following listing.

Listing 6.6 Updating dynamic property on an index

```
PUT cars_with_custom_settings/_settings
{
  "settings": {
    "number_of_replicas": 2
  }
}
```

The `number_of_replicas` property is dynamic, so it doesn't matter if the index is live; the property is applied instantly.

> **NOTE** Elasticsearch does not allow us to change the shard count once the index is operational. There is a simple, yet valid reason for this: a document's home is derived by this routing function: `shard_home = hash(doc_ID) % number_of_shards`. The routing function is dependent on the number of shards, so modifying the shard count (i.e., changing the `number_of_shards`) would break the routing function because existing documents might be misplaced or incorrectly assigned to a shard.

If we want to reconfigure indexes with new settings, we must carry out a few steps:

1. Close the current index (the index cannot support read/write operations).
2. Create a new index with the new settings.
3. Migrate the data from the old index to the new index (reindexing operation).
4. Repoint the alias to the new index (assuming the index has an existing alias).

We have seen reindexing operations in the last chapter (Section 5.7) in action. We also have a section on reindexing data with zero downtime discussed in section 6.2.3.

Fetching configurational settings is a straightforward job: simply issue a `GET` request.

Listing 6.7 Fetching the settings of an index

```
GET cars_with_custom_settings/_settings
```

We can also fetch settings of multiple indexes by using comma-separated indexes or wildcard patterns on the index names. The following listing shows how to do this (the code is available with the book's files).

Listing 6.8 Fetching the settings of multiple indexes at once

```
GET cars1,cars2,cars3/_settings
GET cars*/_settings
```

Fetches multiple index settings

Fetches the settings for the index identified with a wildcard (*)